

Needs Statement: LLVM Visualization System

Purpose

This project provides functional need statements for a compiler visualization system. It enables programmers to observe how high-level code (C++, Rust, or Go) is transformed into LLVM intermediate representation (IR) and ultimately into assembly. The tool provides insights into how header files, language constructs, and compiler optimizations affect the compilation process.

The Program Analyzer Functional Need Statement

The LLVM visualization system is a tool to help programmers and students understand the compilation pipeline and optimization effects in system languages.

1. Program Analyzer Functions

- 1.1 Analyze an existing program written in C++, Rust, or Go to determine its compilation stages.
- 1.2 Visualize LLVM IR generation from source code.
- 1.3 Show transformations during optimization passes.
- 1.4 Produce the final assembly output.
- 1.5 Allow exploration of the effects of header files or expressions (e.g., `constexpr` in C++).
- 1.6 Provide graphical visualizations of compilation steps.
- 1.7 Implement the project using LLVM libraries.

2. Counting Specifications

- 2.1 Count functions, variables, and headers included in the compilation.
- 2.2 Count optimization passes applied and their effects (e.g., reduction in instruction count).
- 2.3 Ignore comments and whitespace in source when analyzing.

3. Analyzer Specifications

- 3.1 Size metrics: LOC of input program; Number of functions and headers; Number of generated IR instructions; LOC of generated assembly.
- 3.2 Complexity metrics: Track optimization passes applied; Compare before/after IR complexity; Compare before/after assembly size.
- 3.3 Dependency metrics: Header dependency graph; Function call graph within IR.

4. Documentation Specifications

- 4.1 Design documentation must describe the architecture and modules (frontend, IR visualizer, assembly generator).
- 4.2 User manual must describe how to load code, run compilation, and view visualizations.
- 4.3 Software test results must show whether the tool correctly generates IR, assembly, and reports metrics for C++, Rust, and Go inputs.

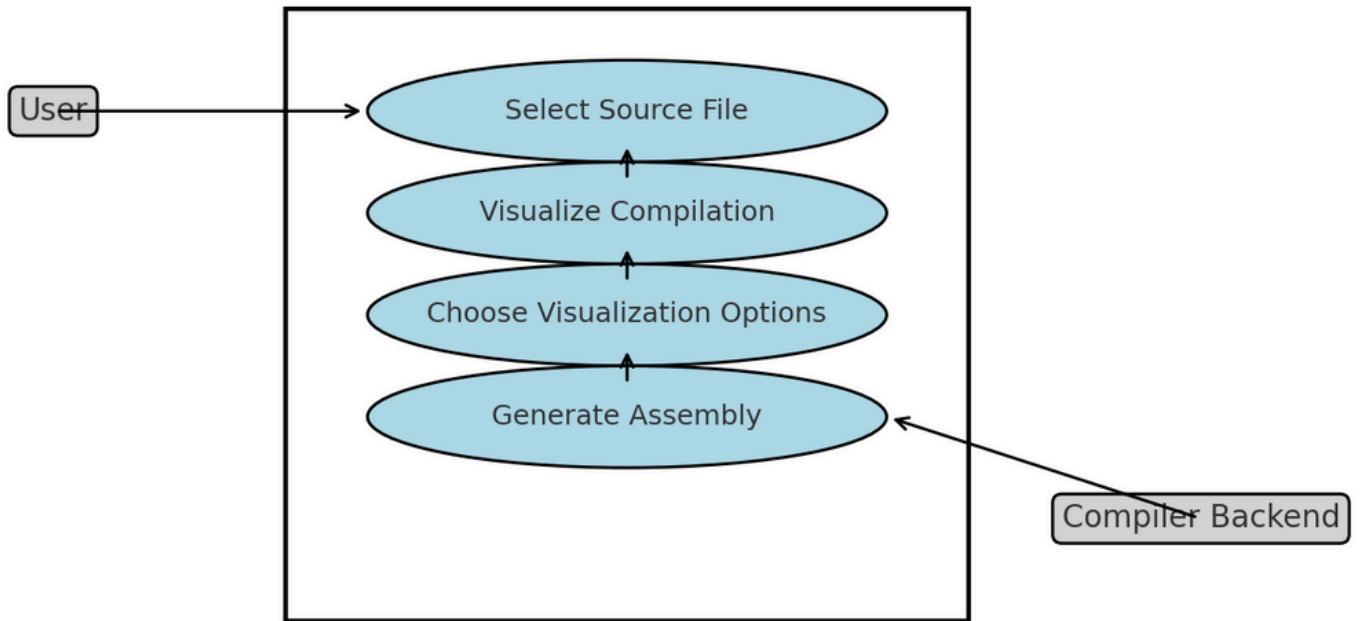
Use Case: Visualize LLVM Compilation

Related use cases:

- Generalization of: Visualize compilation of C++, Rust, or Go code
- Specialization of: View assembly representation

Actor actions	System responses
1. User selects source code file (C++, Rust, or Go)	2. System loads the file.
3. User requests 'Visualize Compilation'.	4. Compiler front-end converts code into LLVM IR.
5. User chooses visualization options	6. System displays intermediate representation and transformations
7. User requests assembly output.	8. System generates and displays assembly code.

LLVM Visualization System



[illegible]